Day 2 – Hour 1: React State Management (useState**) & Synthetic Events**

1. Theory & Core Concepts

A. Props vs. Local State

- **Props (Properties):** Immutable, read-only data passed downward from a parent component to a child. Child components cannot modify incoming props.
- **State:** Dynamic, mutable data managed locally inside a component. Whenever state changes via its setter function, React automatically re-renders the component to update the UI.

Feature	**Props**	**State**
Origin	Passed from parent component	Declared within the component
Mutability	**Immutable** (Read-Only)	**Mutable** via updater function
Re-rendering	Re-renders only when parent passes new props	Automatically triggers component re-render

B. Anatomy of the useState **Hook** React provides the useState hook to declare local component state. It returns an array of two items using array destructuring:

JavaScript

```
const [stateVariable, setStateVariable] = useState(initialValue);
```

- stateVariable: The current value of the state.
- setStateVariable: The updater function used to update the state value and trigger a re-render.
- initialValue: The initial state value set during the first render.

C. Direct Mutation Pitfalls vs. Functional Updates

- **Direct Mutation Fails:** Writing count = count + 1 mutates the variable in place without notifying React's Virtual DOM engine, so no re-rendering occurs.
- **Setter Invocation:** Always invoke the setter: setCount(count + 1).
- **Functional Updater (**prev => prev + 1**):** When the next state depends on the previous state, pass a callback to guarantee working with the latest state snapshot during batched asynchronous updates.

D. Synthetic Event Handling (onClick**)**

React wraps native browser events inside cross-browser **Synthetic Events** using camelCase syntax:

JavaScript

```
// Inline arrow function
<button onClick={() => setCount(prev => prev + 1)}>Like</button>
```

NGIT/KMEC

```
// Named event handler reference
<button onClick={handleReset}>Reset</button>
```

2. Hands-On Lab: React Exercise 2 (Like Button)

Problem Statement: Create a Like Button application using the useState Hook. Display the title 'Like Button Application' along with the current number of likes. Initially, the likes count should be 0. Clicking the **Like** button should increase the count by one, while clicking the **Reset** button should reset the count back to zero. Use event handling and state management to update the UI dynamically.

Project Directory Structure:

Plaintext

```
like-button-app/
├── src/
|   ├── components/
|   |   └── LikeButton.jsx   # Stateful Like Button Component
|   ├── App.jsx            # Root Component
|   ├── App.css            # Component Styling
|   └── main.jsx           # Application Entry Point
├── tests/
|   └── likeButton.spec.js   # Automated Playwright Verification Suite
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Create the Stateful Child Component (src/components/LikeButton.jsx**)**

JavaScript

```
// src/components/LikeButton.jsx
import React, { useState } from 'react';

function LikeButton() {
  // 1. Declare state initialized to 0
  const [likes, setLikes] = useState(0);

  // 2. Increment handler using functional updater
  const handleLike = () => {
    setLikes((prevLikes) => prevLikes + 1);
  };

  // 3. Reset handler
  const handleReset = () => {
    setLikes(0);
  };

  return (
    <div className="like-container">
      <h1 className="title">Like Button Application</h1>
      <p className="likes-count">
        Current Likes: <span className="count-value">{likes}</span>
```

NGIT/KMEC

```
    </p>

    <div className="button-group">
      <button className="btn btn-like" onClick={handleLike}>
        Like
      </button>
      <button className="btn btn-reset" onClick={handleReset}>
        Reset
      </button>
    </div>
   </div>
  );
}

export default LikeButton;
```

Step 2: Render Component in Root (src/App.jsx**)**

```
JavaScript
// src/App.jsx
import React from 'react';
import LikeButton from './components/LikeButton';
import './App.css';

function App() {
  return (
    <div className="app-root">
      <LikeButton />
    </div>
  );
}

export default App;
```

Step 3: Add Styling (src/App.css**)**

```
CSS
/* src/App.css */
.app-root {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f8fafc;
  font-family: Arial, sans-serif;
}

.like-container {
  background-color: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
```

NGIT/KMEC

```
  padding: 2.5rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  text-align: center;
  width: 320px;
}

.title {
  font-size: 1.5rem;
  color: #1e293b;
  margin-bottom: 1.25rem;
}

.likes-count {
  font-size: 1.1rem;
  color: #475569;
  margin-bottom: 1.5rem;
}

.count-value {
  font-weight: bold;
  color: #2563eb;
}

.button-group {
  display: flex;
  justify-content: center;
  gap: 1rem;
}

.btn {
  padding: 0.6rem 1.25rem;
  font-size: 1rem;
  font-weight: 600;
  border-radius: 6px;
  border: none;
  cursor: pointer;
  transition: background-color 0.2s ease;
}

.btn-like {
  background-color: #2563eb;
  color: #ffffff;
}

.btn-like:hover {
  background-color: #1d4ed8;
}

.btn-reset {
  background-color: #e2e8f0;
  color: #334155;
```

NGIT/KMEC

```
}

.btn-reset:hover {
  background-color: #cbd5e1;
}
```

5. Execution Steps

1. **Scaffold Project:**

```
Bash
npm create vite@latest like-button-app -- --template react
cd like-button-app
npm install
```

2. **Mount Code:** Place LikeButton.jsx in src/components/, update src/App.jsx, and add src/App.css.

3. **Start Local Server:**

```
Bash
npm run dev
```

4. **Execute Test Suite:**

```
Bash
npx playwright test tests/likeButton.spec.js
```

Day 2 – Hour 2: Controlled Components & Form Handling
1. Theory & Core Concepts
A. Controlled vs. Uncontrolled Components

- **Uncontrolled Components:** Form data is handled directly by the browser DOM itself via references (useRef). Data is pulled from the DOM only when needed (e.g., during form submission).
- **Controlled Components:** Form inputs derive their current value directly from React state (value={state}), and update that state on every keystroke via change handlers (onChange). React serves as the **single source of truth**.

Feature	**Controlled Component (Recommended)**	**Uncontrolled Component**
Source of Truth	React State (useState)	Browser DOM nodes (ref)
Value Binding	Bound to state variable (value={...})	Read from DOM on demand (ref.current.value)

NGIT/KMEC

Feature	Controlled Component (Recommended)	Uncontrolled Component
Instant Validation	Easy (validates on every keystroke via state)	Difficult (requires direct DOM queries)
UI Synchronization	Dynamic and automatic on state update	Static until explicitly read from the DOM

B. Form Submission & e.preventDefault() In standard HTML, submitting a form sends an HTTP request and triggers a full browser reload. In Single Page Applications (SPAs), e.preventDefault() is invoked inside the submit handler to prevent page refresh, allowing React state to manage submission and dynamic UI updates in memory.

C. Managing Multi-Input Form State Immutably Instead of declaring separate useState hooks for every single input field, manage related inputs inside a single state object using dynamic object keys [e.target.name] and the spread operator ...:

JavaScript

```
const [formData, setFormData] = useState({
  name: '',
  employeeId: '',
  department: '',
  designation: '',
  salary: ''
});

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({
    ...prev,
    [name]: value // Dynamic property computed update
  }));
};
```

2. Hands-On Lab: React Exercise 3 (Employee Registration Form)

Problem Statement: Create an Employee Registration Form using controlled components. Accept **Employee Name**, **Employee ID**, **Department**, **Designation**, and **Salary** from the user. Manage all input fields using the useState Hook. When the **Add Employee** button is clicked, display the entered employee information below the form in a well-formatted section. The form should update the displayed details dynamically.

Project Directory Structure:

Plaintext

```
employee-form-app/
 ├── src/
|   ├── components/
|   |   └── EmployeeForm.jsx    # Controlled Form & Submitted Card View
|   ├── App.jsx               # Main Application Layout
```

NGIT/KMEC

```
|   ├── App.css              # Form & Card Layout Styles
|   └── main.jsx             # Vite React Entry Point
├── tests/
|   └── employeeForm.spec.js    # Automated Playwright Verification Suite
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Create the Controlled Form Component (src/components/EmployeeForm.jsx**)**

JavaScript

```
// src/components/EmployeeForm.jsx
import React, { useState } from 'react';

function EmployeeForm() {
  // 1. State for form input fields
  const [formData, setFormData] = useState({
    name: '',
    employeeId: '',
    department: '',
    designation: '',
    salary: ''
  });

  // 2. State for holding submitted employee details
  const [submittedData, setSubmittedData] = useState(null);

  // 3. Dynamic input change handler
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value
    }));
  };

  // 4. Form submission handler
  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmittedData(formData);
    // Reset form fields
    setFormData({
      name: '',
      employeeId: '',
      department: '',
      designation: '',
      salary: ''
    });
  };

  return (
```

NGIT/KMEC

```
<div className="form-container">
 <h1 className="form-heading">Employee Registration Form</h1>

 <form onSubmit={handleSubmit} className="registration-form">
  <div className="form-group">
   <label htmlFor="name">Employee Name:</label>
   <input
    type="text"
    id="name"
    name="name"
    placeholder="Enter employee name"
    value={formData.name}
    onChange={handleChange}
    required
   />
  </div>

  <div className="form-group">
   <label htmlFor="employeeId">Employee ID:</label>
   <input
    type="text"
    id="employeeId"
    name="employeeId"
    placeholder="Enter employee ID"
    value={formData.employeeId}
    onChange={handleChange}
    required
   />
  </div>

  <div className="form-group">
   <label htmlFor="department">Department:</label>
   <input
    type="text"
    id="department"
    name="department"
    placeholder="Enter department"
    value={formData.department}
    onChange={handleChange}
    required
   />
  </div>

  <div className="form-group">
   <label htmlFor="designation">Designation:</label>
   <input
    type="text"
    id="designation"
    name="designation"
    placeholder="Enter designation"
    value={formData.designation}
```

NGIT/KMEC

```
          onChange={handleChange}
          required
        />
      </div>

      <div className="form-group">
        <label htmlFor="salary">Salary:</label>
        <input
          type="text"
          id="salary"
          name="salary"
          placeholder="Enter salary"
          value={formData.salary}
          onChange={handleChange}
          required
        />
      </div>

      <button type="submit" className="btn-submit">
        Add Employee
      </button>
    </form>

    {/* Submitted Details Display Section */}
    {submittedData && (
      <div className="submitted-details">
        <h2>Registered Employee Details</h2>
        <p><strong>Name:</strong> <span className="display-
name">{submittedData.name}</span></p>
        <p><strong>Employee ID:</strong> <span className="display-
id">{submittedData.employeeId}</span></p>
        <p><strong>Department:</strong> <span className="display-
dept">{submittedData.department}</span></p>
        <p><strong>Designation:</strong> <span className="display-
desig">{submittedData.designation}</span></p>
        <p><strong>Salary:</strong> <span className="display-
salary">{submittedData.salary}</span></p>
      </div>
    )}
  </div>
 );
}

export default EmployeeForm;
```

Step 2: Update App Component (src/App.jsx**)**

JavaScript

```
// src/App.jsx
import React from 'react';
import EmployeeForm from './components/EmployeeForm';
```

NGIT/KMEC

```
import './App.css';

function App() {
  return (
    <div className="app-root">
      <EmployeeForm />
    </div>
  );
}

export default App;
```

Step 3: Add Styling (src/App.css**)**

CSS

```
/* src/App.css */
.app-root {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background-color: #f1f5f9;
  font-family: Arial, sans-serif;
  padding: 2rem 0;
}

.form-container {
  background-color: #ffffff;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  padding: 2rem;
  width: 100%;
  max-width: 450px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

.form-heading {
  text-align: center;
  color: #0f172a;
  font-size: 1.5rem;
  margin-bottom: 1.5rem;
}

.registration-form {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.form-group {
  display: flex;
```

NGIT/KMEC

```
  flex-direction: column;
  text-align: left;
  gap: 0.35rem;
}

.form-group label {
  font-weight: 600;
  color: #334155;
  font-size: 0.9rem;
}

.form-group input {
  padding: 0.6rem;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 0.95rem;
  outline: none;
}

.form-group input:focus {
  border-color: #2563eb;
}

.btn-submit {
  margin-top: 0.5rem;
  padding: 0.75rem;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s ease;
}

.btn-submit:hover {
  background-color: #1d4ed8;
}

.submitted-details {
  margin-top: 2rem;
  padding: 1.25rem;
  background-color: #f8fafc;
  border-left: 4px solid #2563eb;
  border-radius: 4px;
  text-align: left;
}

.submitted-details h2 {
  margin-top: 0;
```

NGIT/KMEC

```
  font-size: 1.2rem;
  color: #1e293b;
  margin-bottom: 0.75rem;
}

.submitted-details p {
  margin: 0.4rem 0;
  color: #475569;
  font-size: 0.95rem;
}
```

Day 2 – Hour 3: Dynamic Lists & Client-Side CRUD Operations

1. Theory & Core Concepts

A. Managing Array State Immutably In React, array state must never be modified directly (e.g., using .push(), .splice(), or .pop()) because direct mutations bypass React's Virtual DOM reconciliation engine and do not trigger a UI update. Always produce a new array reference using the ES6 spread operator (...) or non-mutating methods like .filter() and .map().

- **Adding an Item:**

JavaScript

```
setStudents((prev) => [...prev, newStudent]);
```

- **Deleting an Item by Unique Identifier:**

JavaScript

```
setStudents((prev) => prev.filter((student) => student.rollNumber !== targetRollNumber));
```

B. Conditional UI Rendering When array state is empty (e.g., students.length === 0), conditional rendering provides a fallback message (such as *"No Students Found"*) rather than an empty table.

JavaScript

```
{students.length === 0 ? (
  <p className="no-students">No Students Found</p>
) : (
  <table className="student-table">...</table>
)}
```

C. The Significance of the key **Prop in Dynamic Lists** When rendering lists via .map(), assign a unique, stable property (like rollNumber) to the key attribute. This enables React's diffing algorithm to track item additions, re-orderings, and deletions accurately without re-rendering the entire list.

2. Hands-On Lab: React Exercise 7 (Student Management System)

Problem Statement: Create a Student Management System using the useState Hook. Accept **Student Name**, **Roll Number**, and **Branch** from the user. Initially display 'No Students Found'. Clicking **Add Student** should insert the student into a table. Each row should contain a **Delete** button to remove that student. The UI should update automatically whenever records are added or deleted.

Project Directory Structure:

NGIT/KMEC

Plaintext
```
student-management-app/
├── src/
│   ├── components/
│   │   └── StudentManager.jsx    # Stateful CRUD & Table Component
│   ├── App.jsx                   # Root Application View
│   ├── App.css                   # Responsive Table & Form Styling
│   └── main.jsx                  # React Mounting Entry Point
├── tests/
│   └── studentManager.spec.js    # Playwright Verification Suite
├── index.html
└── package.json
```

3. Step-by-Step Code Implementation

Step 1: Create the Student CRUD Component (src/components/StudentManager.jsx**)**

JavaScript
```
// src/components/StudentManager.jsx
import React, { useState } from 'react';

function StudentManager() {
  // 1. Form state for the three controlled input fields
  const [formData, setFormData] = useState({
    name: '',
    rollNumber: '',
    branch: ''
  });

  // 2. State array for managing the collection of students
  const [students, setStudents] = useState([]);

  // 3. Update input state dynamically
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value
    }));
  };

  // 4. Add Student handler (Create)
  const handleAddStudent = (e) => {
    e.preventDefault();
    if (!formData.name || !formData.rollNumber || !formData.branch) return;

    // Append new student immutably
    setStudents((prev) => [...prev, formData]);

    // Reset input fields
```

NGIT/KMEC

```
  setFormData({
    name: '',
    rollNumber: '',
    branch: ''
  });
};

// 5. Delete Student handler (Delete)
const handleDeleteStudent = (rollToDelete) => {
  setStudents((prev) => prev.filter((student) => student.rollNumber !== rollToDelete));
};

return (
  <div className="student-container">
    <h1 className="main-heading">Student Management System</h1>

    {/* Input Form */}
    <form onSubmit={handleAddStudent} className="student-form">
      <div className="input-group">
        <input
          type="text"
          name="name"
          placeholder="Student Name"
          value={formData.name}
          onChange={handleChange}
          required
        />
      </div>
      <div className="input-group">
        <input
          type="text"
          name="rollNumber"
          placeholder="Roll Number"
          value={formData.rollNumber}
          onChange={handleChange}
          required
        />
      </div>
      <div className="input-group">
        <input
          type="text"
          name="branch"
          placeholder="Branch"
          value={formData.branch}
          onChange={handleChange}
          required
        />
      </div>
      <button type="submit" className="btn-add">
        Add Student
      </button>
```

NGIT/KMEC

```
    </form>

    {/* Dynamic List / Empty State View */}
    <div className="records-section">
     {students.length === 0 ? (
       <p className="no-records">No Students Found</p>
     ) : (
       <table className="student-table">
        <thead>
          <tr>
           <th>Student Name</th>
           <th>Roll Number</th>
           <th>Branch</th>
           <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
           <tr key={student.rollNumber} className="student-row">
            <td className="student-name-cell">{student.name}</td>
            <td className="student-roll-cell">{student.rollNumber}</td>
            <td className="student-branch-cell">{student.branch}</td>
            <td>
             <button
              type="button"
              className="btn-delete"
              onClick={() => handleDeleteStudent(student.rollNumber)}
             >
              Delete
             </button>
            </td>
           </tr>
          ))}
        </tbody>
       </table>
     )}
    </div>
   </div>
  );
}

export default StudentManager;
```

Step 2: Update Application Entry (src/App.jsx**)**

JavaScript

```
// src/App.jsx
import React from 'react';
import StudentManager from './components/StudentManager';
import './App.css';
```

NGIT/KMEC

```
function App() {
  return (
    <div className="app-root">
      <StudentManager />
    </div>
  );
}

export default App;
```

Step 3: Add Styling (src/App.css**)**

CSS

```
/* src/App.css */
.app-root {
  display: flex;
  justify-content: center;
  align-items: flex-start;
  min-height: 100vh;
  background-color: #f8fafc;
  font-family: Arial, sans-serif;
  padding: 3rem 1rem;
}

.student-container {
  background-color: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  padding: 2rem;
  width: 100%;
  max-width: 650px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  text-align: center;
}

.main-heading {
  font-size: 1.6rem;
  color: #0f172a;
  margin-bottom: 1.5rem;
}

.student-form {
  display: flex;
  flex-wrap: wrap;
  gap: 0.75rem;
  justify-content: space-between;
  margin-bottom: 2rem;
}

.input-group {
  flex: 1 1 30%;
```

NGIT/KMEC

```
}

.student-form input {
  width: 100%;
  padding: 0.6rem;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 0.95rem;
  box-sizing: border-box;
}

.student-form input:focus {
  outline: none;
  border-color: #2563eb;
}

.btn-add {
  flex: 1 1 100%;
  padding: 0.75rem;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s ease;
}

.btn-add:hover {
  background-color: #1d4ed8;
}

.records-section {
  margin-top: 1.5rem;
}

.no-records {
  color: #64748b;
  font-size: 1.1rem;
  font-style: italic;
  padding: 1.5rem 0;
}

.student-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}

.student-table th,
```

NGIT/KMEC

```
.student-table td {
  padding: 0.75rem 1rem;
  border-bottom: 1px solid #e2e8f0;
}

.student-table th {
  background-color: #f1f5f9;
  color: #334155;
  font-weight: 600;
}

.btn-delete {
  padding: 0.4rem 0.8rem;
  background-color: #ef4444;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  font-size: 0.85rem;
  cursor: pointer;
  transition: background-color 0.2s ease;
}

.btn-delete:hover {
  background-color: #dc2626;
}
```

With Day 2 complete across all 3 hours and exercises, would you like to proceed with the curriculum plan and module breakdown for **Day 3: Lifecycle Hooks (**useEffect**), REST API Integration & Single Page Routing (**react-router-dom**)**?

NGIT/KMEC